

Backend Developer Guide

Complete reference for engineers contributing to **inventory-reservation** — a production-grade NestJS service that prevents overselling under concurrency using a dual-path Redis + database strategy, circuit breaker, and event-driven architecture.

• NestJS 11

• TypeORM + PostgreSQL

• Redis 7 + BullMQ

• DDD · SOLID · EDA

• Circuit Breaker

• Swagger at /api

Quick Start

Prerequisites

SERVICE	VERSION / PORT	NOTES
nvm	—	Node Version Manager — install from nvm-sh/nvm
Node.js	22	Switched via <code>nvm use 22</code>
PostgreSQL	5434	<code>DB_NAME=inventory_reservation</code>
Redis	6381	Used for atomic stock counters and job queue

Environment

```
PORT=3000
DB_HOST=localhost
DB_PORT=5434
DB_USER=inventory
DB_PASS=inventory
DB_NAME=inventory_reservation
REDIS_HOST=localhost
REDIS_PORT=6381
```

Run locally

```
# Switch to Node 22 (install if not present)
nvm install 22
nvm use 22

# Install dependencies
npm install

# Start in watch mode (hot-reload)
npm run start:dev

# Production build + start
npm run build && npm run start

# Interactive simulation script
npm run simulate
```

Tests

```
# Unit tests only
npm test

# Concurrency tests (run serially)
npm run test:concurrency

# Integration tests
npm run test:integration

# All suites with coverage
npm run test:all && npm run test:cov
```



Swagger UI

Once running, full interactive API docs are available at <http://localhost:3000/api>.



Seed inventory before creating reservations

The Redis stock counter must be primed with `POST /inventory` before any reservation can succeed. An unseeded product causes every reservation attempt to return **409 Conflict**.

Tech Stack

CONCERN	CHOICE	WHY
Framework	NestJS 11	Modular DI, decorator-based, first-class TypeScript
ORM	TypeORM 0.3	Repository pattern, entity decorators, migrations
Database	PostgreSQL 15	ACID transactions, SELECT FOR UPDATE for pessimistic locking
Cache / counters	Redis 7 (ioredis)	Atomic Lua script for stock decrement; sub-millisecond throughput
Job queue	BullMQ (bull 4)	Reliable 2-minute expiry jobs with retries and backoff
Events	@nestjs/event-emitter	In-process pub/sub for domain event listeners
Validation	class-validator + class-transformer	DTO validation via decorators, global ValidationPipe
Security	Helmet + ThrottlerModule	HTTP headers hardening, rate limiting (200/min default, 100/min on POST /reservations)
Observability	LoggingInterceptor + Terminus health	Structured request/response logs with X-Request-Id; /health endpoint
Locking	async-mutex	In-process mutex serialises concurrent requests within a Node.js worker
Docs	@nestjs/swagger	Auto-generated OpenAPI spec at /api

API Reference

Reservations

POST

/reservations

Create a reservation — 201 Created · 409 out of stock

GET

/reservations

Paginated list · query: status?, offset, limit

GET /reservations/:id

Single reservation by UUID

POST /reservations/:id/confirm

Confirm — ACTIVE → CONFIRMED

DELETE /reservations/:id

Cancel — ACTIVE → CANCELLED

GET /reservations/:id/logs

Status transition history

Inventory

POST /inventory

Upsert product + prime Redis stock counter

GET /inventory

List all products

GET /inventory/:productId/audit-logs

Paginated inventory mutation history · query: action?, offset, limit

Health

GET /health

Database + Redis health check (Terminus)

GET /health/circuit-breaker

Returns circuit breaker state: CLOSED | OPEN | HALF_OPEN

Rate limiting

ENDPOINT	LIMIT
POST /reservations	100 req / min / IP
All other endpoints	200 req / min / IP
GET /health	Exempt

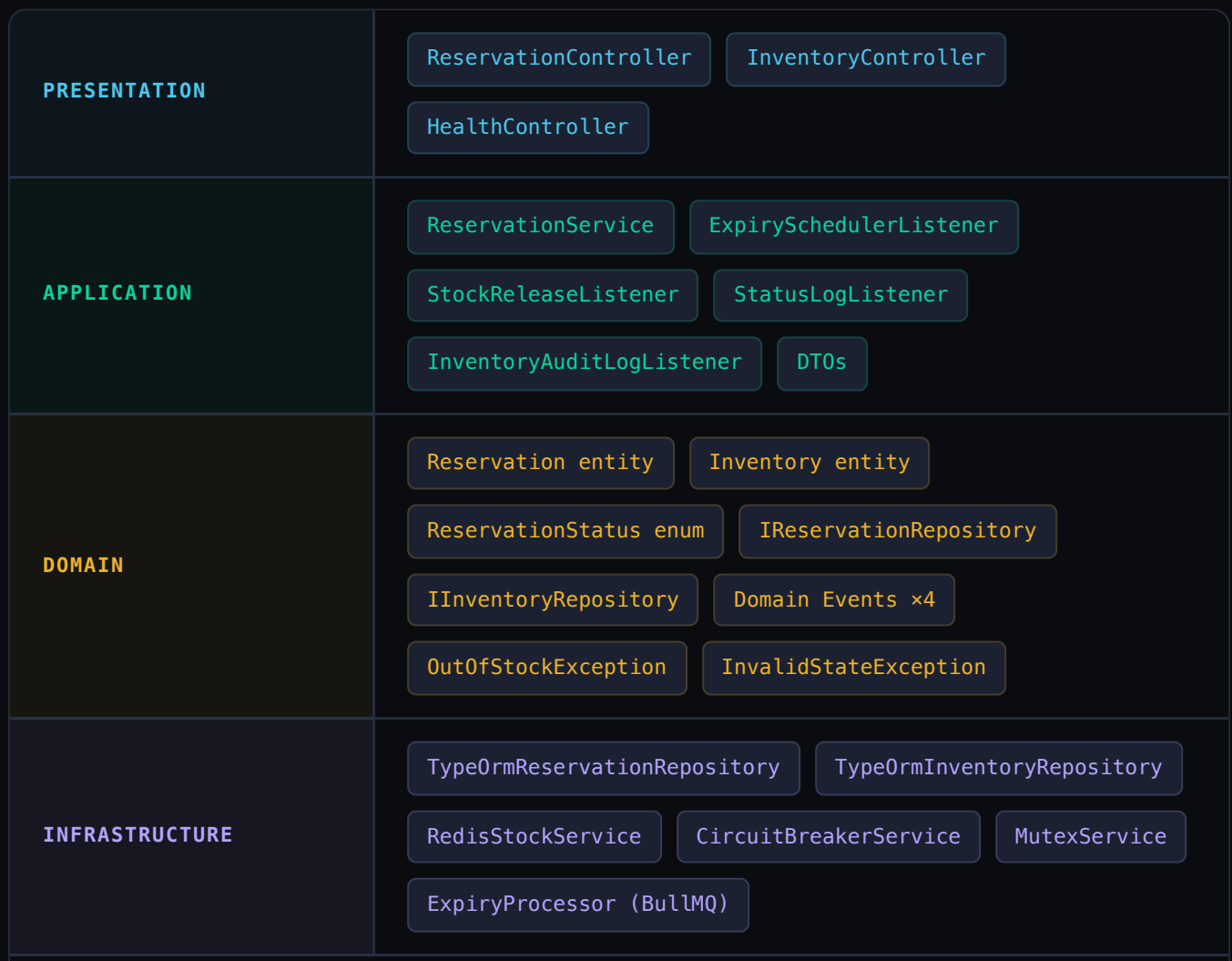
DTOs

```
// POST /reservations body
class CreateReservationDto {
    @IsUUID() @IsNotEmpty() productId: string;
    @IsUUID() @IsNotEmpty() userId: string;
    @IsInt() @Min(1) @Max(100) quantity: number;
}

// POST /inventory body
class UpsertInventoryDto {
    @IsUUID() @IsNotEmpty() productId: string;
    @IsString() @IsNotEmpty() name: string;
    @IsInt() @Min(0) @Max(100000) totalStock: number;
}
```

Layer Map

The backend mirrors the same DDD layering as the frontend. Dependencies only point inward — outer layers import from inner ones, never the reverse.



File Tree

```
src/
├── main.ts                                Bootstrap: Helmet, CORS, Swagger, ValidationPipe, shutdown
├── app.module.ts                          Root module: TypeORM, EventEmitter, Bull, Throttler, all
├──
├── domain/                               Pure business logic – no NestJS / TypeORM imports here
│   ├── entities/
│   │   ├── reservation.entity.ts         Aggregate root with state machine transitions
│   │   ├── inventory.entity.ts           availableStock getter, optimistic locking (version
│   │   ├── reservation-status-log.entity.ts
│   │   └── inventory-audit-log.entity.ts
│   ├── value-objects/
│   │   └── reservation-status.enum.ts
│   ├── events/
│   │   ├── reservation-created.event.ts
│   │   ├── reservation-confirmed.event.ts
│   │   ├── reservation-cancelled.event.ts
│   │   └── reservation-expired.event.ts
│   ├── exceptions/
│   │   ├── out-of-stock.exception.ts     extends ConflictException → 409
│   │   └── invalid-state.exception.ts    extends BadRequestException
│   └── interfaces/
│       ├── reservation-repository.interface.ts
│       └── inventory-repository.interface.ts
├──
├── application/
│   ├── services/
│   │   └── reservation.service.ts        Core orchestrator – dual-path, circuit breaker ro
│   ├── listeners/
│   │   ├── expiry-scheduler.listener.ts  Enqueues BullMQ expiry job on ReservationCre
│   │   ├── stock-release.listener.ts     Releases/confirms stock on Cancel/Expire/Con
│   │   ├── status-log.listener.ts        Writes state-change audit rows
│   │   └── inventory-audit-log.listener.tsWrites inventory mutation history
│   └── dto/
│       └── create-reservation.dto.ts
```

```

|   |   | reservation-response.dto.ts
|   |   | get-reservations-query.dto.ts
|   |   | get-audit-logs-query.dto.ts
|   |   | pagination.dto.ts
|
|   | infrastructure/
|   |   | redis/
|   |   |   | redis-stock.service.ts  Lua atomic decrement, increment, setStock, ping
|   |   |   | redis.module.ts
|   |   |   | redis.constants.ts
|   |   | circuit-breaker/
|   |   |   | circuit-breaker.service.ts  CLOSED/OPEN/HALF_OPEN state machine, 5-failure
|   |   |   | circuit-breaker.module.ts
|   |   | mutex/
|   |   |   | mutex.service.ts  Per-product async-mutex for within-process serial
|   |   | persistence/
|   |   |   | typeorm-reservation.repository.ts
|   |   |   | typeorm-inventory.repository.ts
|   |   | queue/
|   |   |   | expiry.queue.ts
|   |   |   | expiry.processor.ts  Processes expired reservations, dispatches domain
|
|   | presentation/
|   |   | controllers/
|   |   |   | reservation.controller.ts
|   |   |   | inventory.controller.ts
|
|   | health/
|   |   | health.controller.ts
|   |   | health.module.ts
|   |   | redis-health.indicator.ts
|
|   | common/
|   |   | interceptors/
|   |   |   | logging.interceptor.ts  X-Request-Id, structured req/res logs

```

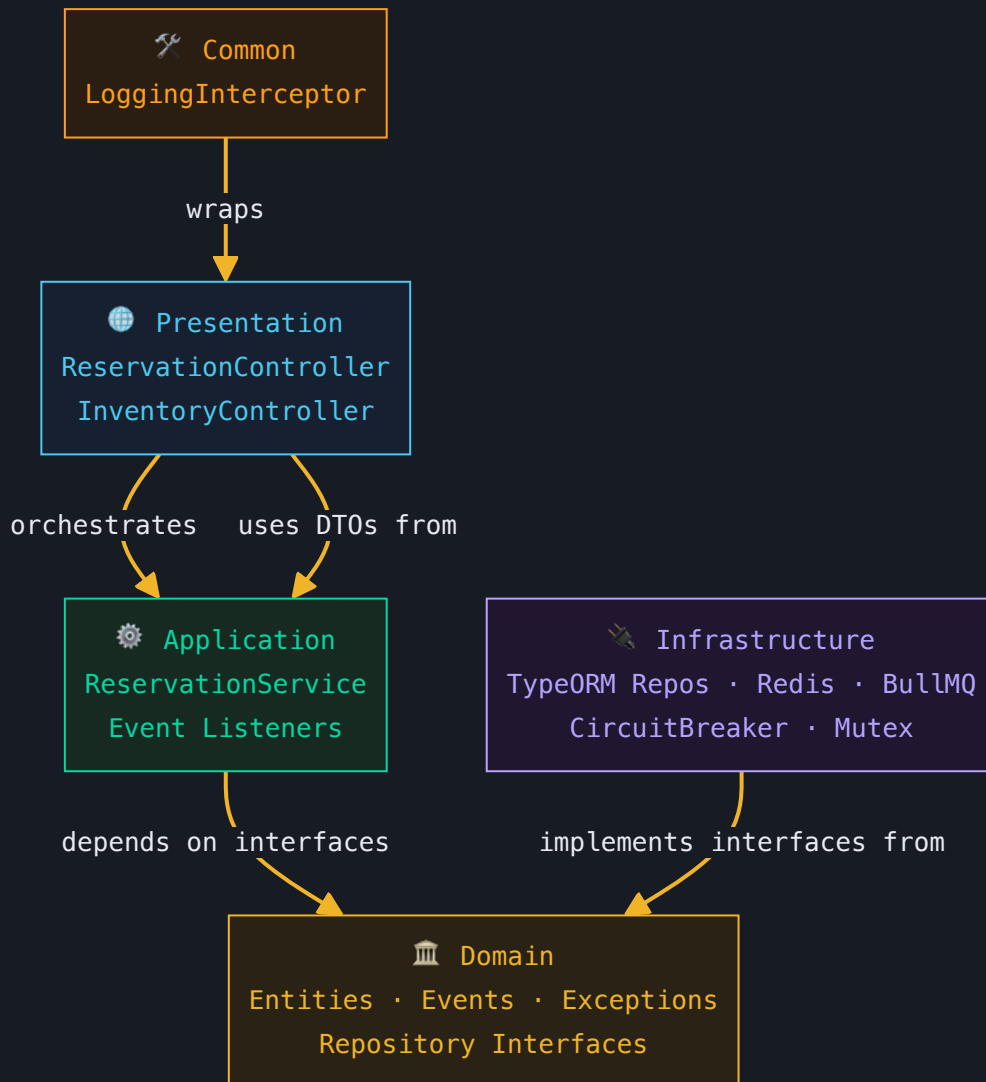
Concept Diagrams

Interactive diagrams illustrating the key architectural patterns of the backend service.

1 — Layer Dependency Graph

Dependencies only flow inward. Infrastructure and Presentation both depend on Domain interfaces, never the other way around.

ARROWS = "DEPENDS ON / IMPORTS FROM"



2 — Reservation State Machine

The `Reservation` aggregate root enforces all transitions. Calling a method on a reservation in an invalid state throws `InvalidStateException` → HTTP 409.

STATEDIAGRAM — RESERVATION LIFECYCLE

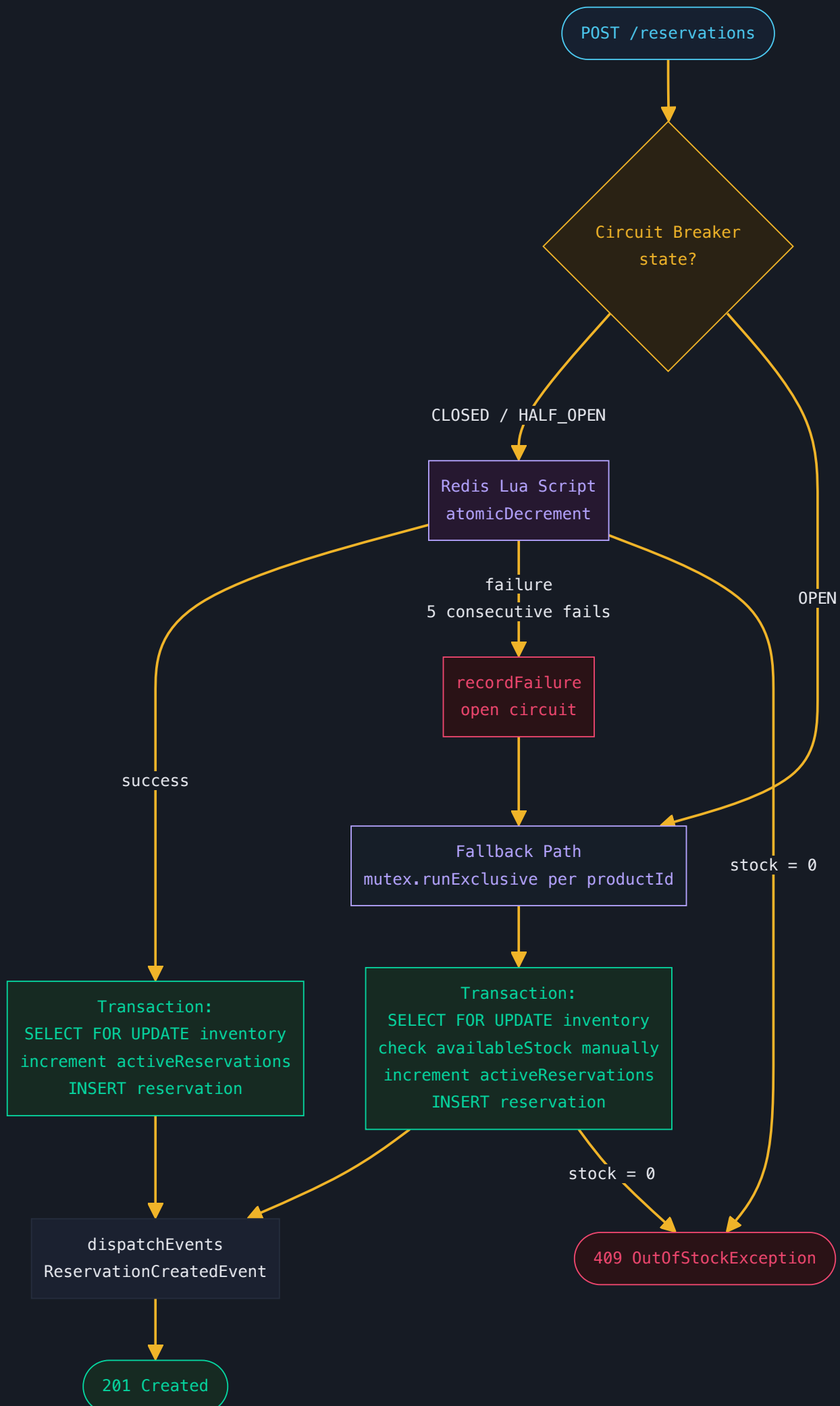
Syntax error in text

mermaid version 10.9.5

3 — Dual-Path Concurrency Strategy

Two layers of protection prevent overselling. Redis Lua is the fast path; Mutex + SELECT FOR UPDATE is the reliable fallback when Redis is unavailable.

FLOWCHART — RESERVATION CREATION ROUTING



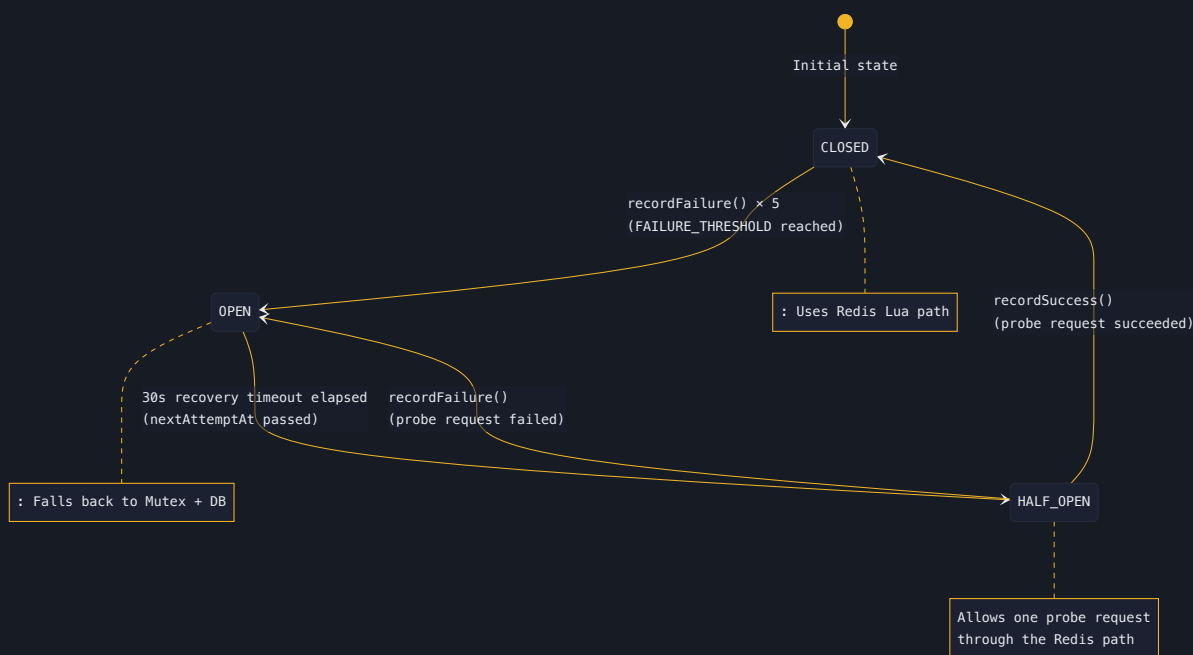
★ Why two layers?

Redis Lua is atomic and fast — ideal for high-concurrency stock decrement. But Redis can fail. When the circuit opens, the fallback uses PostgreSQL's `SELECT FOR UPDATE` combined with an in-process `Mutex` to serialise concurrent requests for the same product within a single Node.js process. Together they guarantee no overselling regardless of Redis availability.

4 — Circuit Breaker State Machine

The circuit breaker routes requests away from Redis when it detects repeated failures, then probes for recovery after a 30-second timeout.

STATEDIAGRAM — CIRCUIT BREAKER



CONSTANT	VALUE	PURPOSE
<code>FAILURE_THRESHOLD</code>	5	Consecutive Redis failures before opening
<code>RECOVERY_TIMEOUT_MS</code>	30 000 ms	How long to stay OPEN before probing

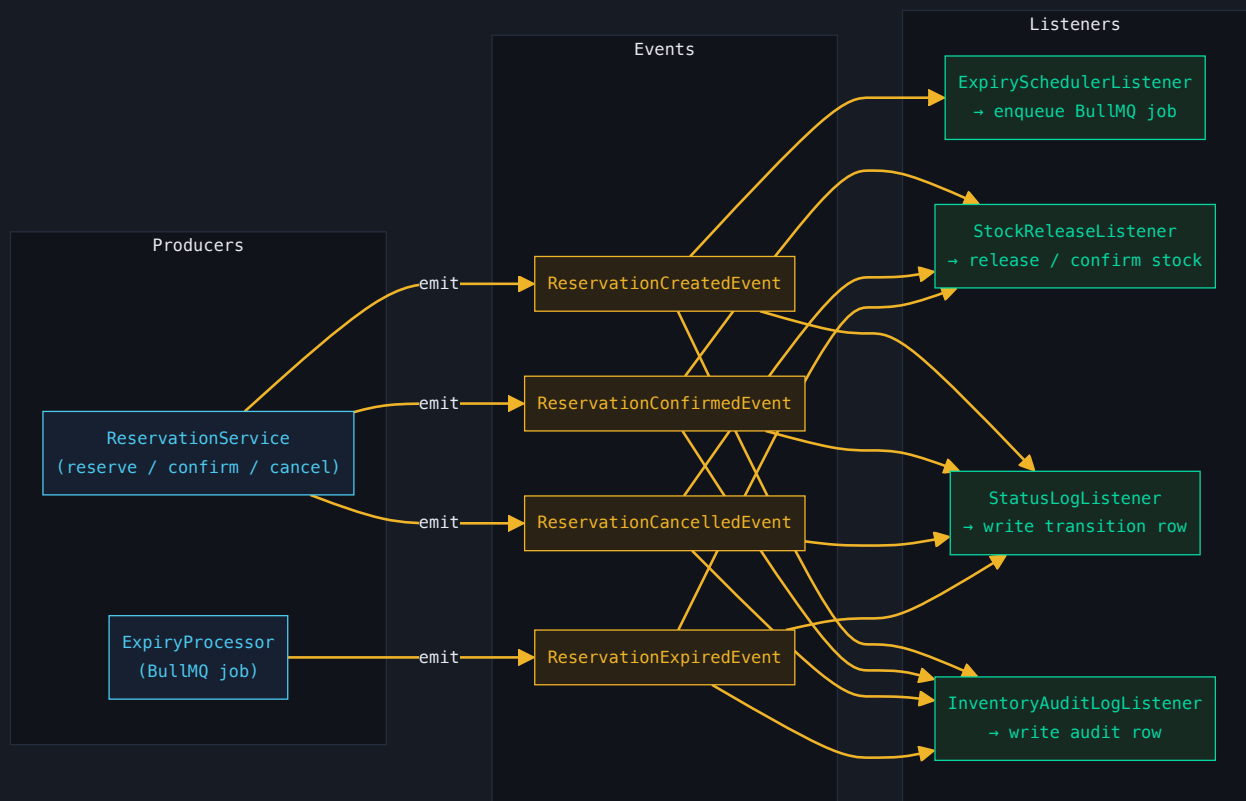
5 — Event-Driven Architecture

After each state transition the `Reservation` entity collects domain events.

`ReservationService.dispatchEvents()` emits them via `EventEmitter2`. Independent

listeners react without coupling to the service.

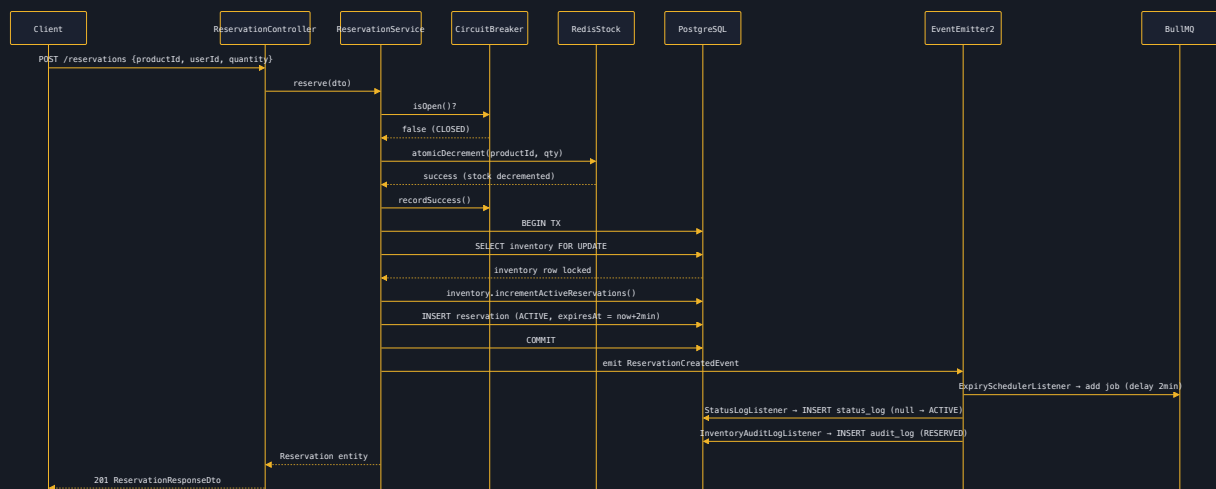
FLOWCHART – EVENT PRODUCERS AND CONSUMERS



6 — Full Request Sequence (POST /reservations)

End-to-end trace of a successful reservation creation through the Redis path.

SEQUENCEDIAGRAM – HAPPY PATH



Domain Layer

Pure business logic with no framework dependencies. This is what makes the system portable and testable in isolation.

Reservation entity — state machine

```
// Factory method – only way to create a reservation
static create(productId, productName, userId, quantity): Reservation {
    // sets status = ACTIVE, expiresAt = now + 2min
    // adds ReservationCreatedEvent to events[]
}

// Transitions – throw InvalidStateException if not ACTIVE
confirm(): void // ACTIVE → CONFIRMED + ReservationConfirmedEvent
cancel(): void // ACTIVE → CANCELLED + ReservationCancelledEvent
expire(): void // ACTIVE → EXPIRED + ReservationExpiredEvent
```

Inventory entity — stock formula

```
// availableStock is always derived, never stored directly
get availableStock(): number {
    return this.totalStock - this.confirmedSales - this.activeReservations;
}

// Called when a reservation becomes ACTIVE
incrementActiveReservations(): void

// Called when ACTIVE → CANCELLED or EXPIRED
decrementActiveReservations(): void

// Called when ACTIVE → CONFIRMED (stock is now sold)
confirmSale(): void // activeReservations-- and confirmedSales++
```

Domain exceptions → HTTP codes

EXCEPTION CLASS	EXTENDS	HTTP	WHEN
OutOfStockException	ConflictException	409	Redis returns ≤ 0 or DB availableStock < requested

EXCEPTION CLASS	EXTENDS	HTTP	WHEN
<code>InvalidStateException</code>	<code>BadRequestException</code>	400	Entity transition called on wrong status

Application Layer

ReservationService — routing logic

```

async reserve(dto: CreateReservationDto): Promise<Reservation> {
  if (this.circuitBreaker.isOpen()) {
    return this.reserveWithFallback(dto); // Mutex + SELECT FOR UPDATE
  }
  try {
    return await this.reserveWithRedis(dto); // Lua atomic decrement
  } catch (e) {
    this.circuitBreaker.recordFailure();
    return this.reserveWithFallback(dto);
  }
}

```

Event listeners

LISTENER	LISTENS TO	DOES
<code>ExpirySchedulerListener</code>	ReservationCreated	Adds BullMQ job with 2-min delay; job ID is idempotent (<code>expiry:{id}</code>)
<code>StockReleaseListener</code>	Cancelled, Expired	DB transaction: decrements activeReservations + Redis increment
<code>StockReleaseListener</code>	Confirmed	DB transaction: <code>confirmSale()</code> — moves activeReservations → confirmedSales
<code>StatusLogListener</code>	All 4 events	Writes a <code>reservation_status_log</code> row
<code>InventoryAuditLogListener</code>	All 4 events	Writes an <code>inventory_audit_log</code> row with quantityDelta

Infrastructure Layer

RedisStockService — Lua atomic decrement

```
-- Atomic: read + compare + decrement in a single Redis operation
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil then return -1 end -- key not seeded
if stock >= tonumber(ARGV[1]) then
    redis.call('DECRBY', KEYS[1], ARGV[1])
    return 1 -- success
end
return 0 -- insufficient stock
```

CircuitBreakerService

METHOD	BEHAVIOUR
<code>isOpen()</code>	Returns <code>true</code> if OPEN and <code>nextAttemptAt</code> hasn't elapsed. Transitions OPEN → HALF_OPEN silently when timeout passes (returns <code>false</code> to allow probe).
<code>recordFailure()</code>	Increments failure count. Opens circuit at threshold. If HALF_OPEN, re-opens immediately.
<code>recordSuccess()</code>	If HALF_OPEN, calls <code>reset()</code> → back to CLOSED.
<code>reset()</code>	Clears failure count and <code>nextAttemptAt</code> , sets state to CLOSED.

MutexService

Maintains a `Map<productId, Mutex>`. `runExclusive(productId, fn)` serialises all concurrent requests for the same product within a single Node.js process, preventing race conditions when Redis is unavailable.

ExpiryProcessor (BullMQ)

```
@Processor(EXPIRY_QUEUE_NAME)
@OnWorkerEvent('completed')
async process(job: Job<ExpiryJobData>) {
    const reservation = await this.reservationRepo.findById(job.data.reservationId);
    try {
```

```

        reservation.expire();           // throws if already CONFIRMED / CANCELLED
        await this.reservationRepo.save(reservation);
        await this.dispatchEvents(reservation);
    } catch (e: InvalidStateException) {
        // Already in terminal state – skip gracefully
    }
}

```

Presentation Layer

Controllers are thin — they validate input via DTOs, call service methods, and map entity responses to DTOs. No business logic here.

ReservationController guards

DECORATOR	APPLIED TO	EFFECT
<code>@Throttle({ default: { limit: 100, ttl: 60000 } })</code>	<code>POST</code> <code>/reservations</code>	100 requests/min per IP
<code>@ParseUUIDPipe()</code>	All <code>:id</code> params	Validates UUID format before hitting service
<code>@HttpCode(201)</code>	POST /reservations	Returns 201 Created (not default 200)

LoggingInterceptor

Applied globally. Generates a `X-Request-Id` UUID header per request. Logs `[id] → METHOD URL` on ingress and `[id] ← METHOD URL STATUS (Xms)` on egress.

Concurrency Strategy Deep Dive

Stock formula

```
availableStock = totalStock - confirmedSales - activeReservations
```

`totalStock` is the total units ever added. `confirmedSales` counts units where payment is finalised. `activeReservations` counts units currently held (but not yet confirmed or released). This formula guarantees no double-counting across concurrent operations.

Layer 1 — Redis Lua script

Atomic read-compare-decrement in a single Redis command. No round-trips. Ideal for high-throughput concurrent reservation creation. Returns `-1` if the key is unseeded (product not in Redis), `0` if insufficient stock, `1` on success.

Layer 2 — Mutex + SELECT FOR UPDATE

When Redis is unavailable (circuit OPEN), a product-scoped Mutex ensures only one coroutine at a time enters the critical section within this Node.js process. Inside the mutex, a `SELECT ... FOR UPDATE` transaction provides cross-process protection at the database level.

Redis rehydration

When Redis recovers (circuit transitions CLOSED via HALF_OPEN), `onRedisRecovery()` queries `getAllAvailableStock()` from the DB and calls `setStock()` for each product, bringing Redis back in sync with the source of truth.



Why not Redis alone?

Redis can fail, restart, or be evicted. If the counter is lost, the DB is still correct. The fallback path using PostgreSQL's MVCC + SELECT FOR UPDATE is the safety net that ensures no overselling even during Redis outages.

Conventions & Rules

Adding a new domain event

1. Create an event class in `src/domain/events/`
2. Add it to the Reservation entity's domain events collection in the appropriate transition method
3. Create a listener in `src/application/listeners/` decorated with `@OnEvent('...')`
4. Register the listener in `app.module.ts` providers



Never modify ReservationService to add side effects

Add a new `@OnEvent()` listener instead. The service is closed for modification (OCP). The event bus is the extension point.

Repository injection

Repositories are injected by Symbol token (`RESERVATION_REPOSITORY`, `INVENTORY_REPOSITORY`), not by class name. This enforces DIP — the service depends on the interface, not the TypeORM implementation.

```
// In app.module.ts
{ provide: RESERVATION_REPOSITORY, useClass: TypeOrmReservationRepository }

// In reservation.service.ts constructor
@Inject(RESERVATION_REPOSITORY) private readonly repo: IReservationRepository
```

Build gate



npm run build must exit 0 before any commit

Runs `nest build` (TypeScript strict). Unit, concurrency, and integration tests should all pass before merging.

inventory-reservation (backend) · NestJS 11 · TypeORM · PostgreSQL · Redis · BullMQ ·
DDD + SOLID + EDA